The Microservices Lie: Why Spring Boot Teams Are Returning to Monoliths in 2025 🔥

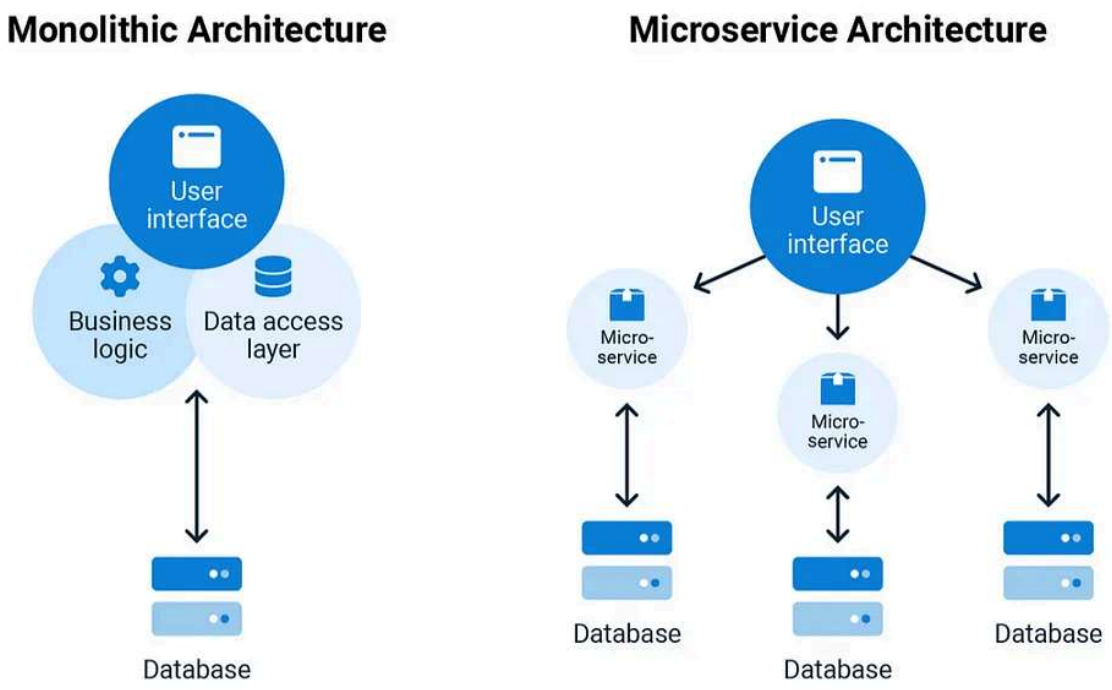
Himanshu · Follow
4 min read · 4 days ago

For years, microservices have been touted as the ultimate architecture for scalable software. Blog posts, conference talks, and tech evangelists all echoed the same mantra: *"Monoliths are dead — long live microservices!"* But in 2025, something unexpected is happening. Teams across the industry — especially those building with **Spring Boot** — are hitting the brakes and asking a bold question:

> **"What if the microservices hype was a lie?"**

This isn't just a hot take. It's a real, measurable trend: developers are moving away from fragmented microservices and consolidating their codebases into what some now call **macrosystems** — modular, maintainable, domain-driven monoliths.

In this article, we'll explore the rise of macrosystems, why teams are turning their backs on microservices, and how **Spring Boot** is powering this new wave of architectural sanity.



Microservices: What We Gained and What We Lost🔍

Microservices came with an enticing promise: break your monolith into smaller, independently deployable pieces. It worked — initially.

✅ Benefits Realized:

- Teams could release features independently.
- Each microservice could scale based on its own demand.
- Services could be written in different languages, tailored to their needs.

But as systems scaled horizontally, cracks started to show.

❌ Pain Points Accumulated:

- **Operational overhead**: Managing dozens (or hundreds) of microservices meant complex CI/CD pipelines, sprawling infrastructure, and countless Kubernetes YAML files.
- **Latency and observability issues**: Every new service added a network hop. Debugging a simple request flow often meant tracing through six to ten services.

- **Code duplication:** Teams recreated utility functions, DTOs, and validation logic across services.
- **Cost:** More services meant more resource consumption and cloud bills.

By 2025, organizations had enough. The question wasn't *"how to scale microservices?"* — it was *"should we even be doing this anymore?"*

Enter Macrosystems: The Rebirth of the Monolith (With a Twist) 🔄

A **macrosystem** is not your 2010s monolith. It's a **modular, domain-aligned**, and **highly maintainable** application that centralizes key functionality without reverting to the Big Ball of Mud.

It brings the best of both worlds:

- The **simplicity** and **coherence** of monoliths.
- The **modular structure**, **scalability**, and **layered architecture** patterns learned from microservices.

In 2025, **Spring Boot** is playing a central role in this architectural renaissance.

How Spring Boot Enables Macrosystems in 2025 ☕

Spring Boot, which was already developer-friendly, has evolved to become the perfect tool for building modular yet consolidated systems. Here's how it's enabling the macrosystem movement:

1. Spring Modularity with Modules and Feature Flags

Teams now organize their macrosystems around **business domains** — think `billing`, `orders`, `auth`, and `inventory` —each encapsulated in separate Spring Boot modules. Tools like **Spring Modulith** and **jigsaw modules** (**Java 17+**) allow clear boundaries without physical separation.

Feature flags and conditional bean loading let teams toggle features or subdomains dynamically — great for gradual rollouts or experimental features.

2. Faster Local Development and CI/CD

Consolidation drastically improves **developer experience**. Instead of spinning up 10+ Docker containers, developers run a single app and get the full context. With Spring Boot DevTools, hot-reloading, and intelligent test slicing, iteration times are faster than ever.

CI/CD pipelines are simpler, too. One deployment artifact. One version. Fewer integration headaches.

3. Unified Observability and Logging

Macrosystems centralize logging, metrics, and tracing. With Spring Boot's native support for Micrometer, OpenTelemetry, and Actuator endpoints, teams now gain **real observability** without stitching together data from dozens of services.

Dashboards are cleaner, alerts more meaningful, and root-cause analysis faster.

4. Security Centralization

Security in microservices meant a tangle of access tokens, service-to-service authentication, and duplicated filters.

Macrosystems simplify this. Spring Security handles everything in one place — session management, RBAC, and even SSO — reducing complexity and enhancing protection.

5. Domain-Driven Design (DDD) with Shared Context

Spring Boot encourages **modular monoliths** through domain-driven design patterns. Each bounded context is maintained separately — sometimes even by different teams — but they all live within a single codebase and deployment unit.

This structure retains team ownership and focus while avoiding the fragmentation of full-blown microservices.

Real-World Migration Strategies 🏗️

The transition from microservices to macrosystems doesn't happen overnight. Teams follow different approaches:

1. Strangler Pattern (Reverse!)

Instead of strangling the monolith, teams now *strangle the microservices* — pulling them into a central, modular app one by one, while keeping interfaces backward-compatible.

2. Shared Libraries to Shared Modules

Common code (like user profiles, error handling, or payment utilities) often lived in shared libraries. These are now integrated as modules within the macrosystem, allowing easier access and versioning.

3. Event Streams to Internal Event Buses

Kafka topics for internal communication are replaced with **in-process event buses**, using Spring Application Events or tools like Axon Framework. It reduces latency and infrastructure complexity.

🚀 Benefits Teams Are Seeing

After consolidation, teams report:

- **30–50% faster release cycles**
- **Significant drop in cloud costs**
- **Fewer production incidents**
- **Better cross-functional collaboration**
- **Easier onboarding for new developers**

One engineering lead from a fintech startup put it best:
"We realized we were spending more time managing services than building value. Consolidation gave us our velocity back."

🧭 When NOT to Consolidate

Not all microservices need to be absorbed. Some still make sense as separate deployments:

- Services with **extremely high throughput** and isolated scaling needs (e.g., real-time data ingestion).
- **Third-party integrations** that are flaky or need sandboxing.
- Legacy systems undergoing **gradual deprecation**.

The goal isn't to abandon microservices entirely, but to be *strategic* about what stays separate and what comes together.

📌 Final Thoughts: The Future is Balanced

2025 marks a more **pragmatic** era in software architecture. Teams aren't religious about microservices anymore — they're strategic. The pendulum has swung from "everything must be a microservice" to "consolidate where it makes sense."

With Spring Boot leading the way, macrosystems are giving teams **control, coherence, and clarity**. It's not a return to the past — it's an evolution forward.

Spring Boot | Software Architecture | Microservices | Monolithic Architecture | Programming

Follow

Written by Himanshu

52 Followers · 3 Following

Hi, I'm Himanshu, a passionate software engineer driven by curiosity and a love for building seamless web experiences while exploring full-stack technologies.

Responses (4)

Neliocacho

What are your thoughts?

Yashbatra
1 day ago

Great insights on spring boot. Really liked the content.

5 Reply

Kaan Atesel
3 days ago

what about scalability power of the microservice? I can scale up one microservice and still keep others as the same?

2 Reply

Arvind Kumar
1 day ago

Every technology comes with its own pros cons and it give the best benefit if applied carefully otherwise it going to hurt alot...i believe same happened with microservices as well....people started creating separate service for each new requirement... more

1 Reply

See all responses

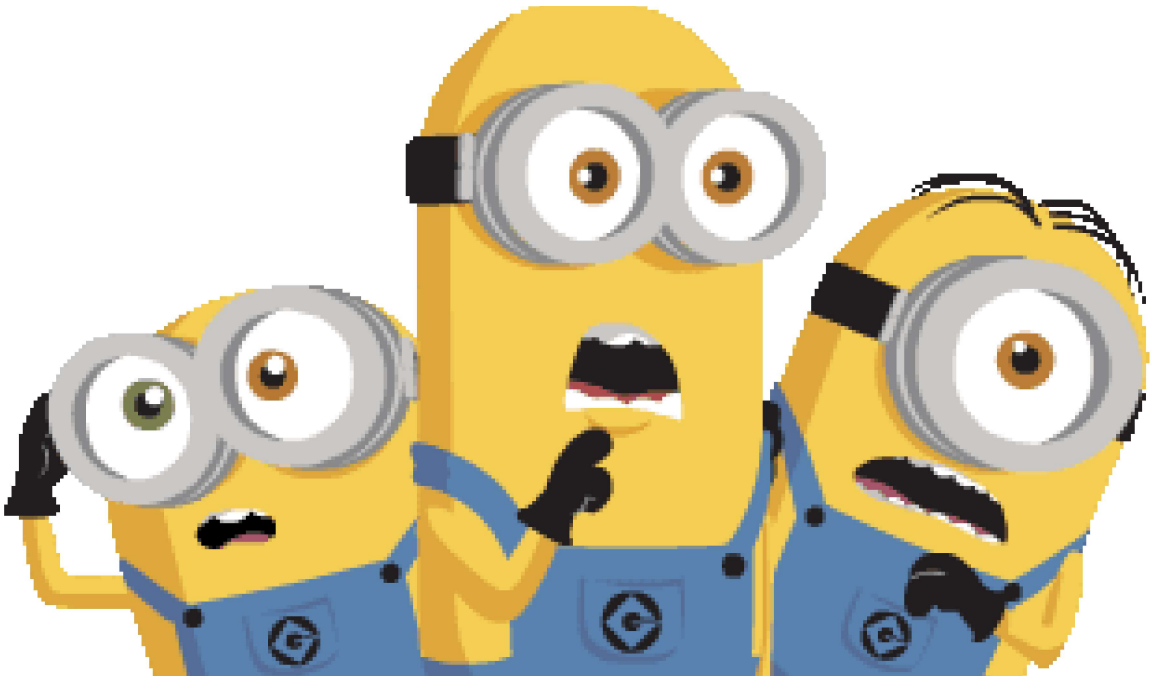
More from Himanshu

Himanshu

Is Spring Boot Too Slow in 2025? Why Startup Times Are Killing Developer Productivity 🐢

Spring Boot has long been a favorite framework for building Java-based microservices and web applications. With its powerful abstraction...

Apr 8 28 1

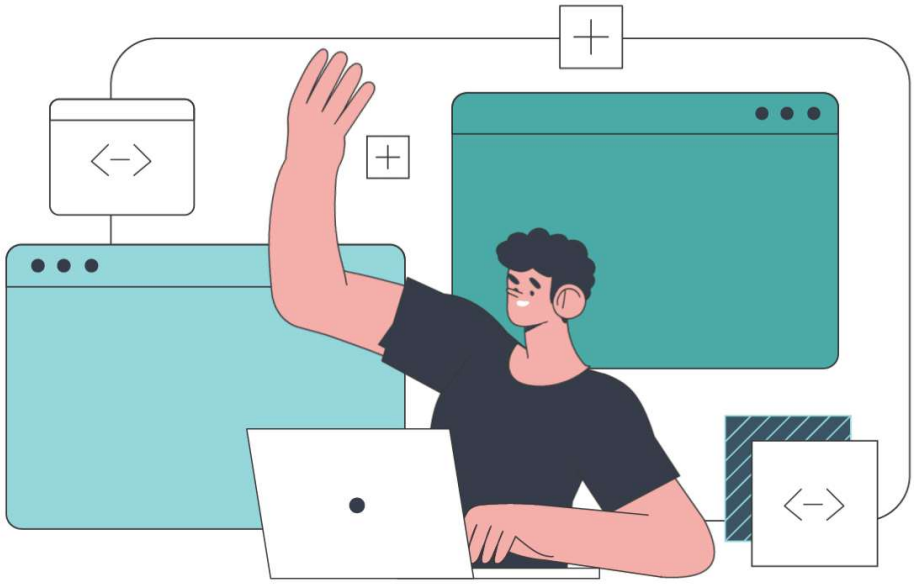


Himanshu

Kafka Is Overkill: The Dark Side of Event-Driven Microservices

In the world of microservices, Kafka has become the golden hammer—wielded by every architect chasing scalability and “modern” design...

4d ago 61 1

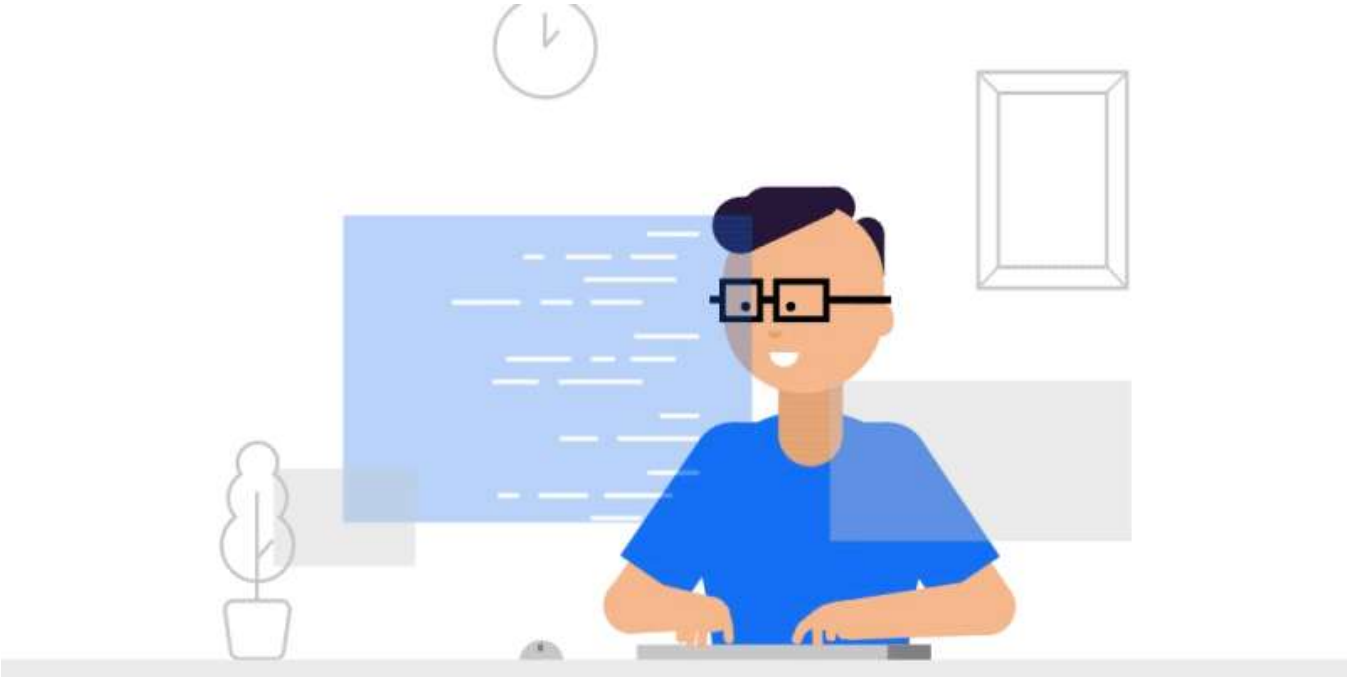


Himanshu

Why Most Spring Boot Apps Fail Without Domain-Driven Design (DDD)

Let’s be honest—most Spring Boot applications are built the wrong way. They start off clean but quickly devolve into tangled service...

Apr 10 7 1



Himanshu

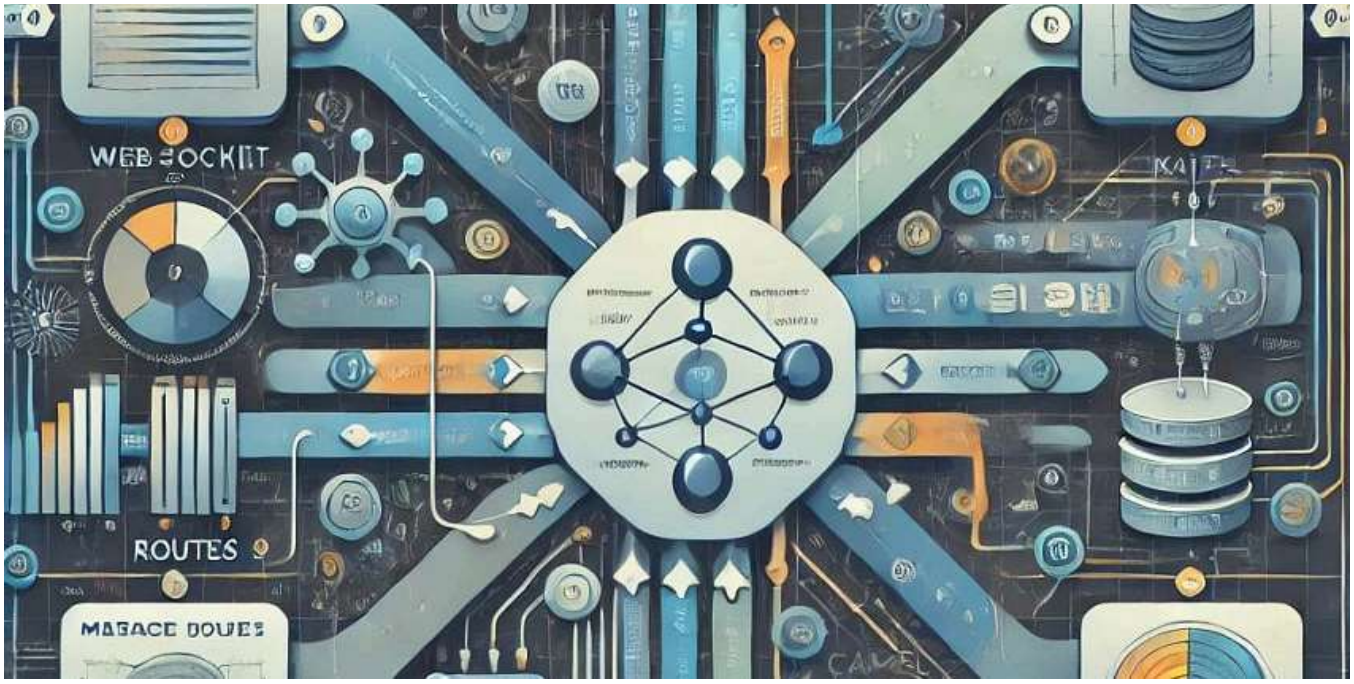
Why Most Spring Boot Apps Get Concurrency Wrong (And How to Fix It)

When it comes to building scalable and high-performance backends with Spring Boot, developers often reach for @Async, thread pools, or even...

6d ago 25

See all from Himanshu

Recommended from Medium



Murat Karakurt

🐫 Apache Camel in Spring Boot

If you've ever had to connect different systems—like moving files, transforming data, or integrating services—then you've probably...

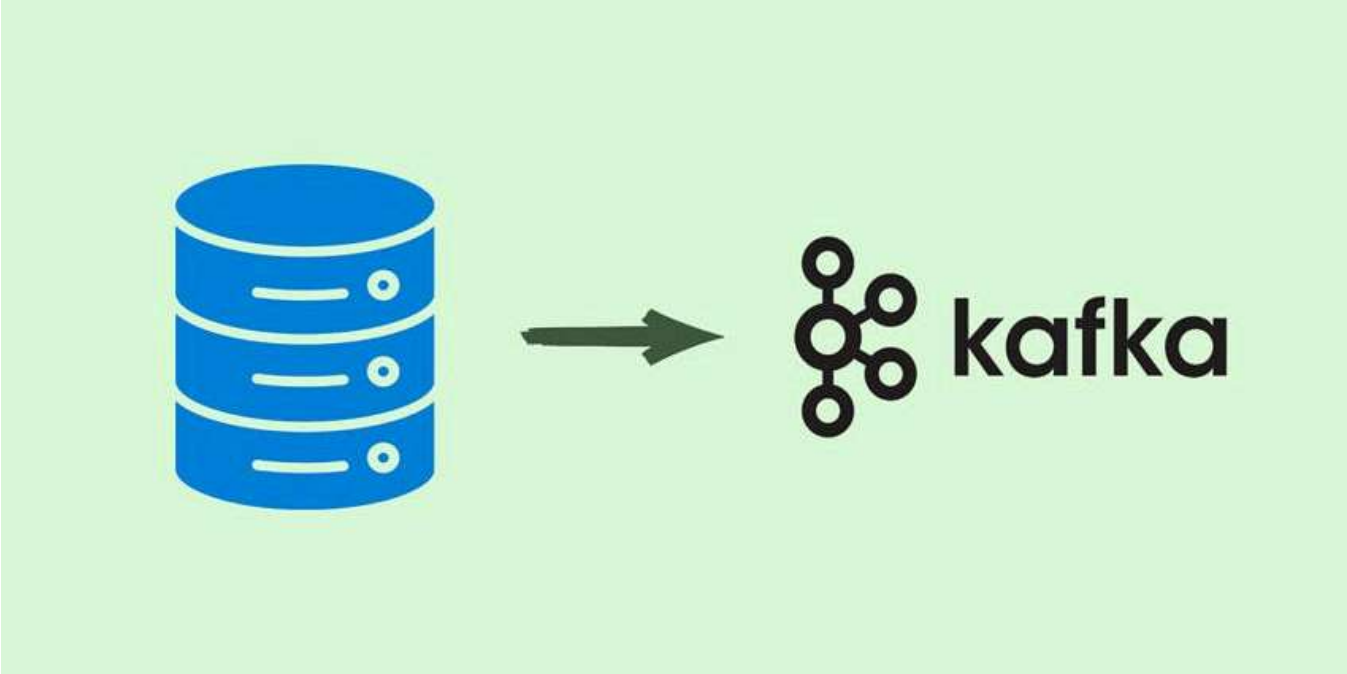
4d ago 131

Sergiy Yevtushenko

Asynchronous Processing in Java with Promises

Traditionally, asynchronous processing is considered complex and error-prone. There are several approaches to address this issue:

5d ago 34 2



The Quantum Yogi

This One Kafka Design Pattern Changed Everything for Us

When we first adopted Kafka, we treated it like a message queue. Producers dumped events, consumers picked them up, and life went on —...

Apr 6 196 15

Practice	Why It Matters
Use `@Transactional` only in service layer	Keeps business and web logic separate
Use method-level transactions	Gives fine control and clarity
Use `@Transactional(readOnly = true)`	Optimizes read-only operations
Avoid `@Transactional` on repositories	Repositories should not control transactions
Keep transactional methods public	Required for Spring proxies
Log or monitor transactional behavior	Helps catch unexpected issues

In Javarevisited by Ramesh Fadatare

🚫 Stop Using @Transactional Everywhere: Understand When You Actually Need It

This article will explain: What Transactional actually does, What Spring Data JPA does for you automatically, When should use...

6d ago 145 1

Hui

2ms vs 500ms—The Shocking Cost of Reflection in Java Object Creation

In Java, the most common way to create an object is simply by using the new keyword:

6d ago 123 7

Oliver Foster

Stream is Awesome, Map is Cool, but Promise Me: Don’t Use toMap()

My article is open to everyone; non-member readers can click this link to read the full text.

5d ago 125 3

See more recommendations